

SUPPLY MANAGEMENT SYSTEM

Task Addendum

Supplier Financial Management & Extended Reporting

Maps to FSD Addendum 22 | ASP.NET Core 8 | EF Core 8 | SQL Server 2022
Version 1.0 | 2026

FSD source	FSD Addendum 22 - Supplier Financial Management & Extended Reporting
Total tasks	9
Total estimated dev days	23 days (~5 working weeks)
Critical priority tasks	3
Total unit test cases	49
Epics	4 epics across supplier ledger, payments, aging, and reporting

Table of Contents

Table of Contents 2

1. FSD Traceability Summary 3

2. Detailed Task Register 4

3. Sequencing Notes..... 11

1. FSD Traceability Summary

Every task maps to a specific FSD Addendum 22 section.

Task ID	Title	Days	FSD Ref	Priority
SFM-001	SupplierLedgerEntries schema and ISupplierLedgerService	3d	Sec 2	Critical
SFM-002	Wire Invoice approval to create a debit ledger entry	2d	Sec 2.2	High
SFM-003	SupplierPayments and SupplierPaymentLines schema and CRUD	3d	Sec 3.1, 3.2	Critical
SFM-004	Payment posting - atomic ledger write and invoice status update	3d	Sec 3.4, 4	Critical
SFM-005	Cheque bounce reversal	2d	Sec 3.3	High
SFM-006	Supplier aging calculation and API	2d	Sec 5	High
SFM-007	Supplier payment and financial reports (5 reports)	3d	Sec 7.1	High
SFM-008	Stock Level Summary report	2d	Sec 7.2	High
SFM-009	PR Fulfillment Status and PR-to-PO Pipeline reports	3d	Sec 7.3, 7.4	High
TOTAL	9 tasks	23d		

2. Detailed Task Register

Epic 1 - Supplier Ledger Foundation (FSD Section 2)

SupplierLedgerEntries entity, running balance logic, and the ledger-write integration into existing Invoice approval flow.

SFM-001	Critical	3d	Sec 2	SupplierLedgerEntries schema and ISupplierLedgerService
---------	----------	----	-------	---

Create the SupplierLedgerEntries entity in finance_schema. Implement ISupplierLedgerService with PostEntryAsync that maintains the running balance per supplier atomically.

Development tasks	Unit test cases
- Add SupplierLedgerEntries entity to FinanceDbContext per FSD Section 2.1 field definitions (14 fields) - EF Core migration: dotnet ef migrations add AddSupplierLedger - Implement ISupplierLedgerService.PostEntryAsync(supplierId, transactionType, referenceType, referenceId, referenceNo, debitAmount, creditAmount, narration) - reads current balance (last entry's balance_after for this supplier), calculates new balance_after, inserts entry atomically - Concurrency safety: row-level lock or serializable scope on the per-supplier balance read-then-write, consistent with Stock Ledger pattern - GET /api/suppliers/{id}/ledger - paginated entries with date range filter, ordered by entry_date descending - GET /api/suppliers/{id}/balance - summary: totalDebits, totalCredits, netBalance, availableAdvanceCredit	- PostEntryAsync with debit 10,000 on a supplier with balance 0 creates entry with balance_after = 10,000 - PostEntryAsync with credit 5,000 on that supplier creates entry with balance_after = 5,000 - Two concurrent PostEntryAsync calls for the same supplier both succeed with correct sequential balance_after values (no lost update) - GET /api/suppliers/{id}/ledger returns entries in reverse chronological order with running balance - GET /api/suppliers/{id}/balance correctly sums debits, credits, and net balance

Acceptance criteria	- Running balance always accurate under concurrent writes - Append-only - no entries ever updated or deleted - Ledger entries always written in the same transaction as the triggering action
---------------------	---

SFM-002	High	2d	Sec 2.2	Wire Invoice approval to create a debit ledger entry
---------	------	----	---------	--

Extend the existing Invoice approval flow to automatically create a SupplierLedgerEntry (debit = invoice_amount) when an invoice passes 3-way match and is approved.

Development tasks	Unit test cases
In the existing Invoice approval handler (FSD v3 Section 10), after approval succeeds, call ISupplierLedgerService.PostEntryAsync within the same transaction: transaction_type = INVOICE_APPROVED, debit_amount = invoice.amount, reference_id = invoice.id	- Approving an invoice for 50,000 creates a SupplierLedgerEntry with debit_amount = 50,000 and correct balance_after - Approving a CreditNote for 5,000 creates a credit entry reducing the supplier balance

• Ensure the ledger write is part of the approval transaction - if the ledger write fails, the entire approval rolls back • Wire the Addendum 8A credit note and debit note approval flows similarly: CreditNote creates a credit entry; DebitNote creates a debit entry	• Simulated ledger-write failure during invoice approval rolls back the entire approval transaction • Invoice approval for a supplier with no prior ledger entries creates the first entry with balance_after = invoice_amount
Acceptance criteria	• Every invoice approval automatically creates a ledger debit • Credit/debit notes from Addendum 8A correctly create offsetting entries • Ledger write and approval are genuinely atomic

Epic 2 - Payment Processing (FSD Sections 3, 4)

SupplierPayments, SupplierPaymentLines, invoice allocation, payment posting, and the cheque-bounce reversal flow.

SFM-003	Critical	3d	Sec 3.1, 3.2	SupplierPayments and SupplierPaymentLines schema and CRUD
---------	----------	----	--------------	---

Create the payment entities and build the create/read/approve lifecycle, including multi-invoice allocation on payment lines.

Development tasks	Unit test cases
- Add SupplierPayments (18 fields) and SupplierPaymentLines (6 fields) entities to FinanceDbContext - EF Core migration: dotnet ef migrations add AddSupplierPayments - POST /api/supplier-payments - create DRAFT payment with optional payment lines allocating against invoices from GET /api/suppliers/{id}/outstanding-invoices - Validation: sum of payment_line.allocated_amount must equal total_amount; each allocated_amount must not exceed the invoice's outstanding_amount - Conditional field validation per payment_method: bank_account required for BANK_TRANSFER/ONLINE_WIRE; cheque_no and cheque_date required for CHEQUE - GET /api/supplier-payments - paginated list with filters: supplier, status, method, date range - GET /api/supplier-payments/{id} - full detail with payment lines and linked invoice numbers - POST /api/supplier-payments/{id}/approve - Finance Manager role required; transitions DRAFT to APPROVED - POST /api/supplier-payments/{id}/cancel - cancel DRAFT or APPROVED (not POSTED)	- POST payment with 3 invoice lines summing to total_amount succeeds - POST payment with lines summing to less than total_amount returns 400 - POST with method = CHEQUE and missing cheque_no returns 400 - POST with method = BANK_TRANSFER and missing bank_account returns 400 - Approve by Finance Officer (not Manager) returns 403 - Cancel a POSTED payment returns 422 - GET outstanding-invoices returns only invoices with payment_status != FULLY_PAID for the given supplier

Acceptance criteria	- Multi-invoice allocation correctly validated - Conditional method fields enforced - Approval restricted to Finance Manager role
---------------------	---

SFM-004	Critical	3d	Sec 3.4, 4	Payment posting - atomic ledger write and invoice status update
---------	----------	----	------------	---

Implement POST /api/supplier-payments/{id}/post that writes the ledger entry and updates each allocated invoice's paid_amount and payment_status in one transaction.

Development tasks	Unit test cases
POST /api/supplier-payments/{id}/post - validates status = APPROVED; within one EF Core transaction: call	Posting a 50,000 payment against one 50,000 invoice: Invoice.paid_amount = 50,000,

ISupplierLedgerService.PostEntryAsync (credit_amount = total_amount), then for each payment line update Invoice.paid_amount += allocated_amount and recalculate Invoice.payment_status per FSD Section 4 - Payment status derivation: paid_amount = 0 then UNPAID; 0 < paid_amount < invoice_amount then PARTIALLY_PAID; paid_amount = invoice_amount then FULLY_PAID; paid_amount > invoice_amount then OVERPAID (system alert) - Set SupplierPayment.posted_at = now, status = POSTED - For ADVANCE_PAYMENT type: create SupplierAdvancePayments record with available_balance = total_amount, for future offset against invoices - For PURCHASE_RETURN_SETTLEMENT type: reduce the source CreditNote's remaining_credit by allocated_amount	payment_status = FULLY_PAID, supplier balance reduced by 50,000 - Posting a 30,000 payment against a 50,000 invoice: paid_amount = 30,000, payment_status = PARTIALLY_PAID - Posting against two invoices (20,000 + 30,000): both invoices updated correctly, one ledger entry for total 50,000 - ADVANCE_PAYMENT creates SupplierAdvancePayments with correct available_balance - Simulated failure after ledger write but before invoice update rolls back everything - Posting a payment that would make paid_amount > invoice_amount triggers OVERPAID system alert
---	---

Acceptance criteria	- Ledger write and all invoice updates genuinely atomic - Payment status correctly derived for all 4 states - Advance payments correctly tracked for future offset
----------------------------	--

SFM-005	High	2d	Sec 3.3	Cheque bounce reversal
----------------	-------------	-----------	----------------	-------------------------------

Implement the bounce flow for CHEQUE payments: reversing ledger entry, invoice paid_amount rollback, and payment status update.

Development tasks	Unit test cases
- POST /api/supplier-payments/{id}/bounce - validates status = POSTED and payment_method = CHEQUE - Within one transaction: write a reversing SupplierLedgerEntry (debit_amount = payment's total_amount, narration = 'Cheque bounced'), decrement Invoice.paid_amount by each line's allocated_amount, recalculate each Invoice.payment_status, set SupplierPayment.status = BOUNCED - Bounce is only available for CHEQUE method - other methods have no bounce flow	- Bouncing a 50,000 cheque payment creates a debit ledger entry for 50,000, restoring the supplier balance - Invoice that was FULLY_PAID reverts to UNPAID after bounce (paid_amount back to 0) - Invoice that was PARTIALLY_PAID by the cheque and another prior payment correctly reverts to the pre-cheque paid_amount - Bouncing a non-CHEQUE payment returns 422 - Bouncing a non-POSTED payment returns 422

Acceptance criteria	- Reversal is complete and atomic - Invoice status correctly recalculated after the bounce reversal - Only CHEQUE payments can be bounced
----------------------------	---

Epic 3 - Supplier Aging (FSD Section 5)

Aging calculation engine and the per-supplier aging breakdown API.

SFM-006	High	2d	Sec 5	Supplier aging calculation and API
---------	------	----	-------	------------------------------------

Build the aging calculation that buckets outstanding invoice amounts into current/30/60/90/120+ brackets based on due_date, and the API endpoint returning the breakdown.

Development tasks	Unit test cases
- GET /api/suppliers/{id}/aging - calculates aging per outstanding invoice (payment_status != FULLY_PAID) based on days since due_date vs today - Bucket logic: Current (0-30), 31-60, 61-90, 91-120, 120+ days overdue - Age on outstanding_amount (invoice_amount - paid_amount), not full invoice_amount - partially-paid invoices age only on the unpaid portion - Response: per-bucket totals + grand total + list of invoices per bucket for drill-down - GET /api/reports/supplier-aging - cross-supplier aging report: one summary row per supplier with bucket totals, plus a grand total row across all suppliers	- Invoice due 15 days ago with outstanding 10,000 appears in the Current bucket - Invoice due 45 days ago with outstanding 20,000 appears in the 31-60 bucket - Partially-paid invoice (amount 50,000, paid 30,000) due 75 days ago: 20,000 appears in 61-90 bucket, not 50,000 - FULLY_PAID invoice does not appear in any aging bucket - Cross-supplier report correctly aggregates all suppliers with outstanding payables

Acceptance criteria	- Aging uses outstanding_amount, not full invoice_amount - All 5 buckets correctly computed - Cross-supplier report provides meaningful summary
---------------------	---

Epic 4 - Extended Reporting Suite (FSD Section 7)

8 new reports: supplier payment reports, stock level summary, and PR fulfillment tracking.

SFM-007	High	3d	Sec 7.1	Supplier payment and financial reports (5 reports)
---------	------	----	---------	--

Implement the 5 supplier payment reports: Payment Register, Supplier Ledger Report, Aging Report, Outstanding Payables, and Payment Method Breakdown.

Development tasks	Unit test cases
- GET /api/reports/supplier-payment-register - all payments with supplier, date, method, status, amount, bank ref filters - GET /api/reports/supplier-ledger - chronological ledger per supplier with running balance, date range filter - GET /api/reports/supplier-aging - cross-supplier aging (built in SFM-006, exposed here as a report endpoint) - GET /api/reports/outstanding-payables - unpaid/partial invoices grouped by supplier with outstanding amount and days overdue - GET /api/reports/payment-method-breakdown - totals by payment method over a date range - All endpoints use read-only ReportsDbContext	- Payment Register correctly filters by supplier, status, method, and date range simultaneously - Supplier Ledger Report running balance matches the live balance from GET /api/suppliers/{id}/balance - Outstanding Payables correctly excludes FULLY_PAID invoices - Payment Method Breakdown totals by method reconcile with the sum of all payments in the date range - All 5 endpoints execute read-only (verified blocked INSERT attempt)

Acceptance criteria	- All 5 payment reports return accurate, paginated data - Ledger report balance reconciles with live balance - Read-only architecture maintained
---------------------	--

SFM-008	High	2d	Sec 7.2	Stock Level Summary report
---------	------	----	---------	----------------------------

Build the stock level summary showing per-product per-warehouse quantities, costs, and value, filterable by warehouse/category/sub-category.

Development tasks	Unit test cases
- GET /api/reports/stock-level-summary - joins InventoryItems with Products, Categories, SubCategories, Warehouses - Columns: product_code, product_name, category, sub_category, warehouse, qty_on_hand, qty_reserved, qty_available, reorder_point, unit_cost, total_value (qty_on_hand * unit_cost) - Filters: warehouse_id, category_id, sub_category_id, search (product name/code) - Grand total row: sum of total_value across all filtered results - Highlight products where qty_available <= reorder_point (below reorder level)	- Report returns correct qty_on_hand, qty_reserved, qty_available for a known product+warehouse combination - Filtering by warehouse returns only products in that warehouse - total_value correctly calculated as qty_on_hand * unit_cost per row - Grand total correctly sums total_value across all rows - Products below reorder level are flagged in the response

Acceptance criteria	- Accurate stock figures matching InventoryItem live data - Category/warehouse filtering works correctly - Reorder-level highlighting functional
----------------------------	--

SFM-009	High	3d	Sec 7.3, 7.4	PR Fulfillment Status and PR-to-PO Pipeline reports
----------------	-------------	-----------	---------------------	--

Build the two PR fulfillment reports: status overview (fulfilled/partial/unfulfilled/cancelled) and the pipeline report showing the furthest downstream document reached.

Development tasks	Unit test cases
- GET /api/reports/pr-fulfillment-status - for each APPROVED PR, compute fulfillment status per FSD Section 7.4: join PR lines to PO lines (via pr_line_id FK), then PO lines to GRNLines (via po_line_id), aggregate qty_received vs ordered_qty - Fulfillment derivation: all lines fully received = FULLY_FULFILLED; some received = PARTIALLY_FULFILLED; no PO at all = UNFULFILLED; PR cancelled = CANCELLED - Columns: pr_number, requester, approval_date, total_lines, fulfilled_lines, pending_lines, fulfillment_percentage - Filters: date range, department, project, fulfillment status - GET /api/reports/pr-to-po-pipeline - for each approved PR, determine the furthest document in the chain: Awaiting Quotation / Quotation Sent / Quotation Awarded / PO Created / PO Sent / GRN Received / Invoice Paid / Material Disbursed. Uses trace_id (Addendum 21) when available, falls back to FK chain traversal otherwise - Summary counts at the top: total PRs, fully fulfilled count, partially count, unfulfilled count	- PR with all lines having PO lines where GRN qty_received >= ordered_qty shows FULLY_FULFILLED - PR with 3 lines, 2 fully received and 1 with no PO shows PARTIALLY_FULFILLED with fulfillment 66% - PR with zero PO references shows UNFULFILLED - Cancelled PR shows CANCELLED and is excluded from fulfillment percentage calculations - Pipeline report correctly shows 'GRN Received' for a PR whose chain has reached that point - Pipeline report correctly uses trace_id when available for chain resolution - Summary counts at top reconcile with the detailed rows below

Acceptance criteria	- Fulfillment status computed accurately from live PO/GRN data, not stored - Pipeline report correctly identifies the furthest downstream document - Summary counts provide at-a-glance procurement health visibility
----------------------------	---

3. Sequencing Notes

Dependencies follow the natural data flow: ledger first, then payments, then aging, then reports.

Dependency	Reason
SFM-001 before everything else	All tasks write to or read from the supplier ledger.
SFM-001 before SFM-002	Invoice approval wiring calls ISupplierLedgerService.
SFM-003 before SFM-004	Posting operates on APPROVED payments created by SFM-003.
SFM-004 before SFM-005	Bounce reverses entries created by the posting logic.
SFM-004 before SFM-006	Aging reads Invoice.paid_amount updated by posting.
SFM-001 through SFM-006 before SFM-007	Reports source data from ledger, payments, and aging.
SFM-008 independent	Stock Level Report reads InventoryItems directly - no dependency on payment tasks.
SFM-009 independent	PR Fulfillment reads PR/PO/GRN data - no dependency on payment tasks. Can be parallelised with SFM-007.

SFM-008 and SFM-009 can be developed in parallel with all other tasks since they read from pre-existing entities (InventoryItems, PRs, POs, GRNs) and have no dependency on the supplier financial entities.

- End of Document -